Name:Maheshwar RG
Regd. No: 23MIS0529
Agile & DevOps Lab Assignment 2 Document

1. Learning Goal

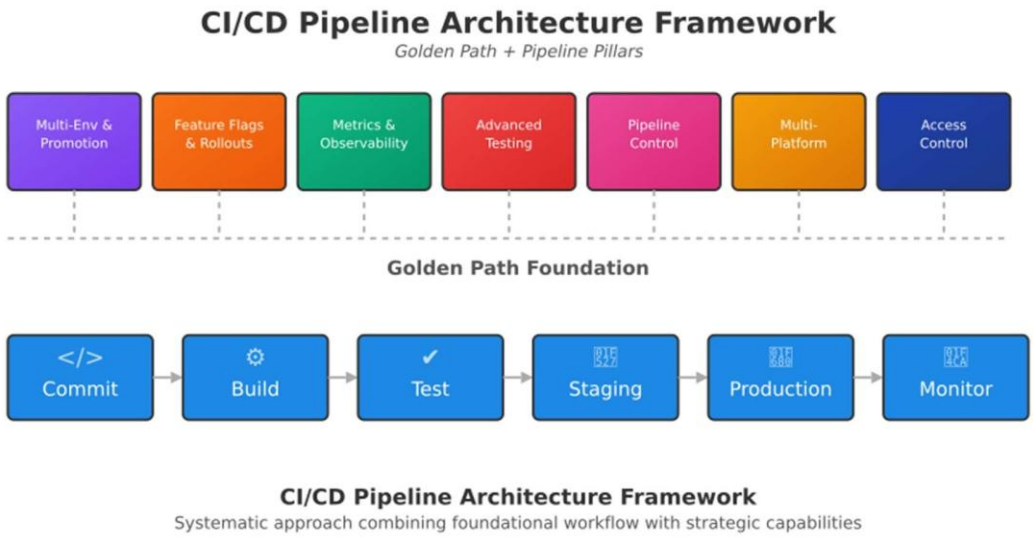
Understand what Jenkins is and run a job successfully

Stepwise Tasks

1. **What is CI/CD?**
   CI/CD stands for Continuous Integration and Continuous Delivery/Deployment, a DevOps practice that automates the process of integrating code changes, testing them, and deploying applications reliably and e iciently
   Real-world example: A developer adds a feature to an e-commerce app and pushes code to GitHub.

   - CI: GitHub Actions automatically runs tests and builds a Docker image.
   - CD: If tests pass, the app deploys to staging. After approval, it goes live on production via Kubernetes.

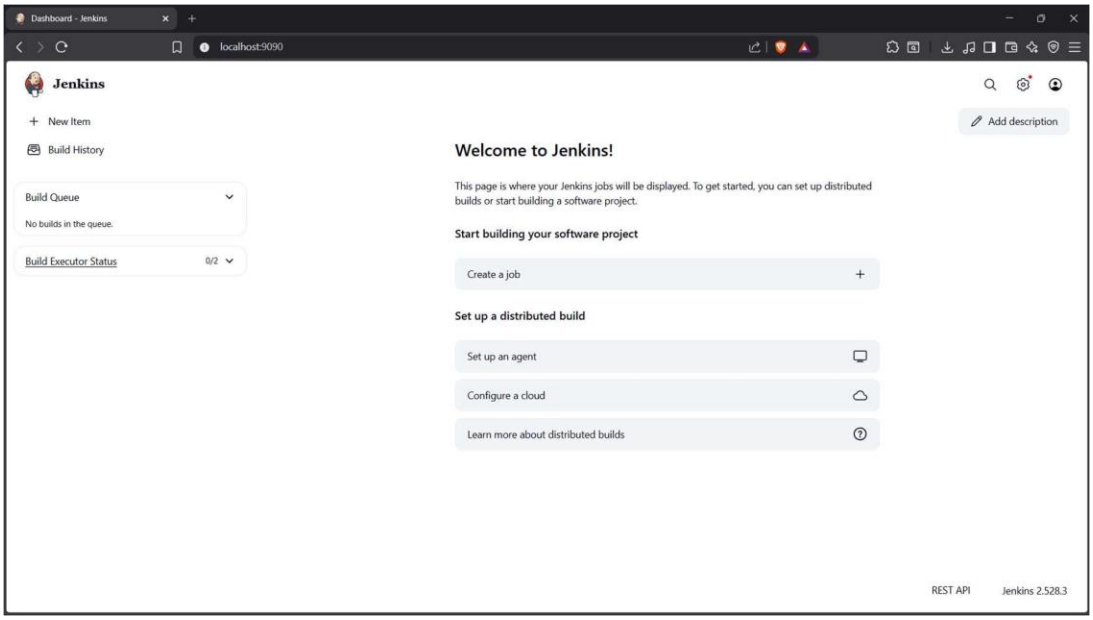
   This ensures fast, safe updates—used by companies like Netflix and Amazon.



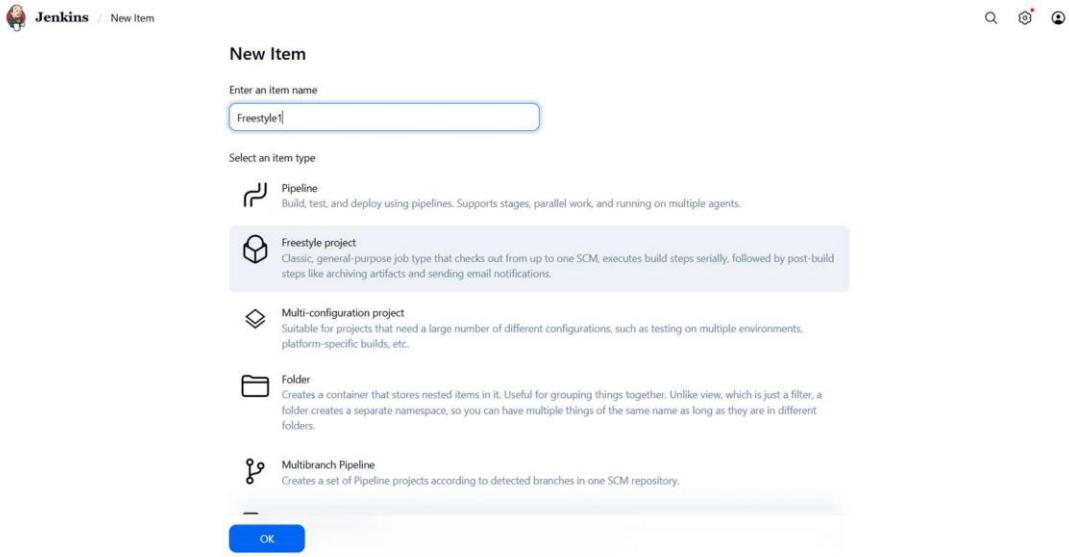
2. **Install Jenkins (Windows / Linux)**

```
SERVICE_NAME: jenkins
        TYPE               : 10  WIN32_OWN_PROCESS
        STATE              : 4  RUNNING
                                (STOPPABLE, NOT_PAUSABLE, ACCEPTS_SHUTDOWN)
        WIN32_EXIT_CODE    : 0  (0x0)
        SERVICE_EXIT_CODE  : 0  (0x0)
        CHECKPOINT         : 0x0
        WAIT_HINT          : 0x0
```

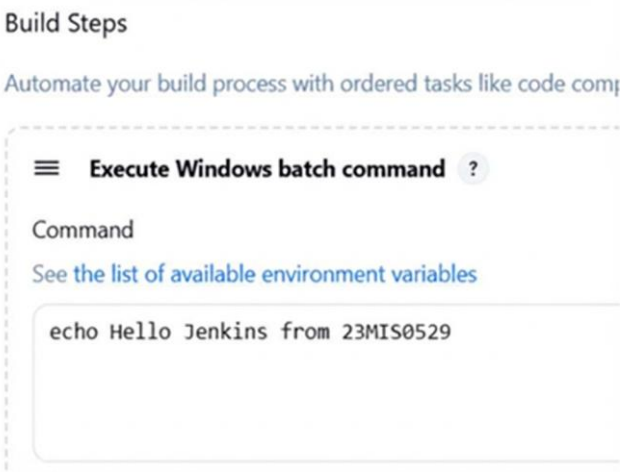
3. **Jenkins UI overview**



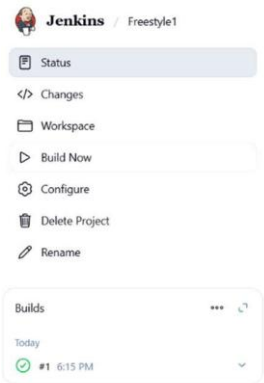
4. **Create Freestyle Job**



5. **Add simple build step:**
   Windows: echo Hello Jenkins



6. **Run job manually** o



7. View:

o Console Output

o Build History

Outcome: A learner can install Jenkins and run a basic job.

2. Learning Goal

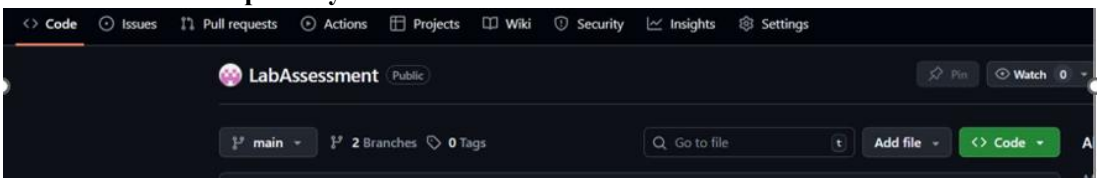
Work with source code and build automation

Stepwise Tasks

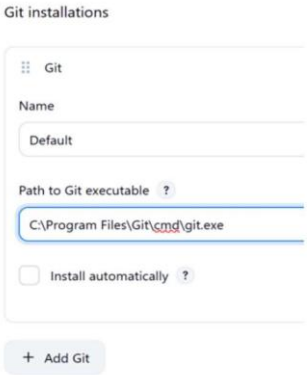
1. **Install Git**

```
C:\Users\Maheshwar>git --version
git version 2.51.0.windows.2
```

2. **Create GitHub repository**

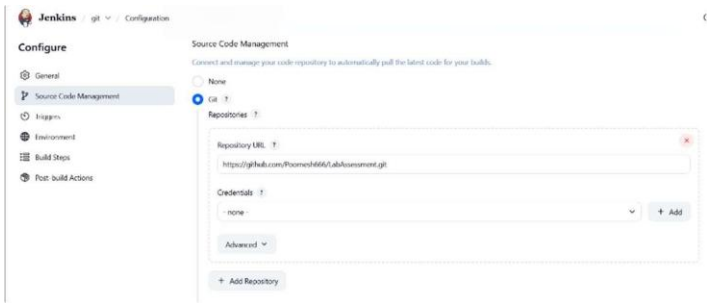


3. **Configure Git in Jenkins**

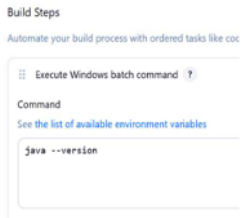


4. **Create Freestyle Job**:

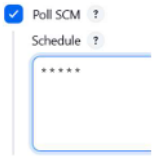
o Source Code Management → Git



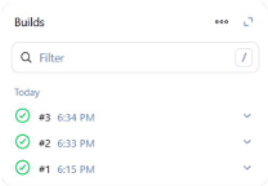
5. Add build step:
o Windows batch



6. Poll SCM



7. Build automatically on code change

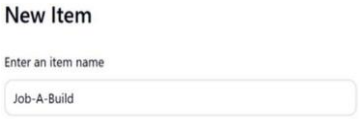


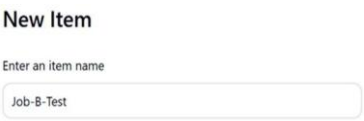
Outcome: Learner connects Jenkins with GitHub.

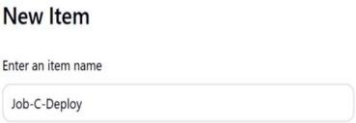
3. Learning Goal: Understand job flow visually

Stepwise Tasks

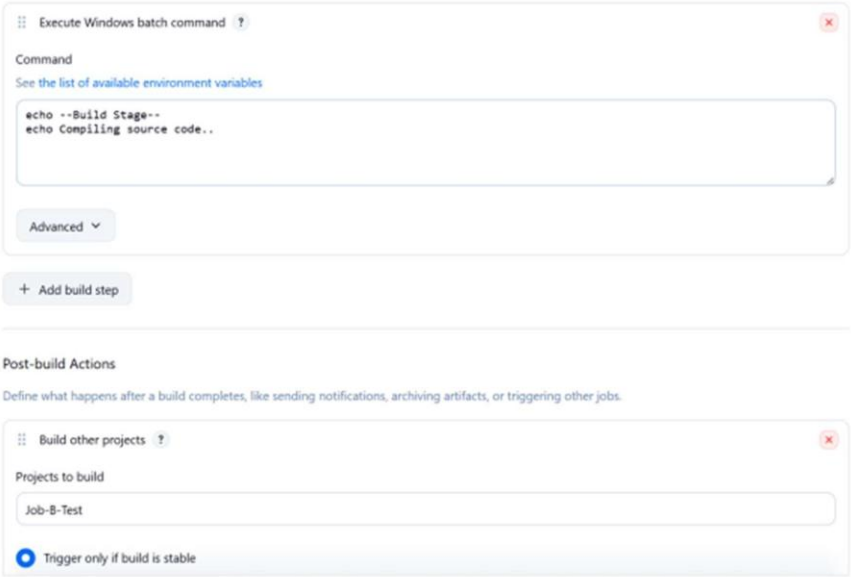
1. **Create multiple Freestyle jobs:**
   o Job-A (Build) o Job-B (Test) o Job-C (Deploy)







2. **Use: o Post-build action → Trigger other projects**



3. **View job chaining visually**

```
Started by user Maheshwar RG
Running as SYSTEM
Building in workspace C:\ProgramData\Jenkins\.jenkins\workspace\Job-A-Build
[Job-A-Build] $ cmd /c call C:\WINDOWS\TEMP\jenkins6303576109228977297.bat

C:\ProgramData\Jenkins\.jenkins\workspace\Job-A-Build>echo --Build Stage--
--Build Stage--

C:\ProgramData\Jenkins\.jenkins\workspace\Job-A-Build>echo Compiling source code..
Compiling source code..

C:\ProgramData\Jenkins\.jenkins\workspace\Job-A-Build>exit 0
Triggering a new build of Job-B-Test
Finished: SUCCESS
```

Console

```
Started by user Maheshwar RG
Running as SYSTEM
Building in workspace C:\ProgramData\Jenkins\.jenkins\workspace\Job-B-Test
[Job-B-Test] $ cmd /c call C:\WINDOWS\TEMP\jenkins10128356484779730585.bat

C:\ProgramData\Jenkins\.jenkins\workspace\Job-B-Test>echo --Test Stage--
--Test Stage--

C:\ProgramData\Jenkins\.jenkins\workspace\Job-B-Test>echo Running test cases
Running test cases

C:\ProgramData\Jenkins\.jenkins\workspace\Job-B-Test>exit 0
Triggering a new build of Job-C-Deploy
Finished: SUCCESS
```

Job-A-Build

Downstream Projects

- Job-B-Test

Permalinks

- Last build (#3), 1 min 29 sec ago
- Last stable build (#3), 1 min 29 sec ago
- Last successful build (#3), 1 min 29 sec ago
- Last completed build (#3), 1 min 29 sec ago

Job-B-Test

Upstream Projects

- Job-A-Build

Downstream Projects

- Job-C-Deploy

Permalinks

- Last build (#3), 1 min 24 sec ago
- Last stable build (#3), 1 min 24 sec ago
- Last successful build (#3), 1 min 24 sec ago
- Last completed build (#3), 1 min 24 sec ago

Job-C-Deploy

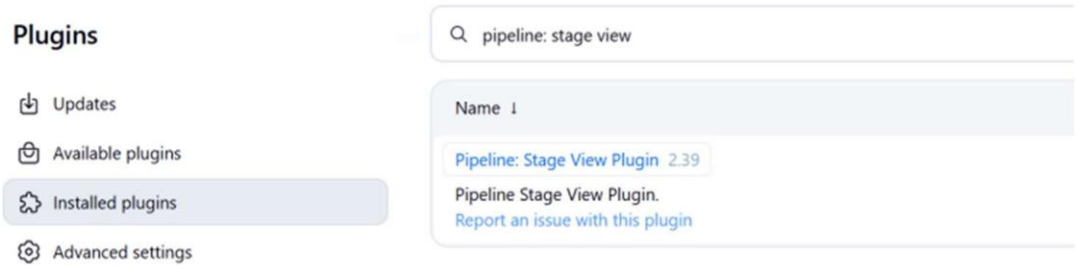
Upstream Projects

- Job-B-Test

Permalinks

- Last build (#3), 1 min 45 sec ago
- Last stable build (#3), 1 min 45 sec ago
- Last successful build (#3), 1 min 45 sec ago
- Last completed build (#3), 1 min 45 sec ago

4. **Install Pipeline Stage View**



5. **Observe stage-wise execution**

Outcome: Learner understands pipeline flow visually.

4. Learning Goal: Use Jenkins Pipeline & Jenkinsfile

Stepwise Tasks

1. What is Jenkins Pipeline?
Jenkins Pipeline is a suite of plugins that enables the automation of continuous delivery (CD) workflows in Jenkins. It allows teams to define the entire software delivery process— such as building, testing, and deploying—as code, using a file called Jenkinsfile. This approach, known as pipeline as code, stores the pipeline definition in version control alongside the application source code, making it easy to review, version, and collaborate on.

A Jenkins Pipeline consists of key components:

- Pipeline: The full workflow, defined in code.
- Stage: A logical phase in the pipeline (e.g., Build, Test, Deploy).
- Step: A single task within a stage (e.g., compiling code, running tests).
- Agent: The machine (node) where the pipeline runs.

Pipelines are durable, pausable, and resumable, meaning they can survive Jenkins restarts and wait for human approval. They support complex workflows like parallel execution, loops, and branching.

2. **Scripted vs Declarative Pipeline**

Feature	Declarative Pipeline	Scripted Pipeline

Syntax	Simplified, YAML-like structure with predefined blocks (pipeline, agent, stages, stage , steps).	Full Groovy scripting with flexible, imperative syntax.
Structure	Strict, predefined structure — required blocks ensure consistency.	Flexible and unconstrained — developers can write custom logic.
SCM Handling	Implicit checkout scm — automatically checks out source code from the repository.	Requires manual checkout — no automatic SCM checkout.
Post Actions	Built-in post block for cleanup, notifications, or actions after pipeline execution.	Implemented using try/catch/finally bl ocks in Groovy.
Stage Restart	Supports restarting from a specific stage — useful for rerunning failed stages without rebuilding.	Does not support restart from stage — entire pipeline must be reexecuted.
Code Validation	Validates pipeline syntax at startup — fails early if syntax errors exist.	No syntax validation at startup — errors detected only during execution.
Use Case	Ideal for simple, standard CI/CD pipelines with clear, predictable workflows.	Better suited for complex, dynamic workflows requiring advanced logic and control.

Readability	High readability and maintainability — easy to understand and share across teams.	Less readable for nonGroovy developers — more verbose and complex.

3. **Create Pipeline Job**

4. **Write pipeline in Jenkins UI**

5. **Add stages:**
   - **Build**
   - **Test**
   - **Deploy**

Sample Task

```
pipeline {
  agent any
  stages {
    stage('Build') { steps { echo 'Build' } }
    stage('Test')  { steps { echo 'Test' } }
    stage('Deploy'){ steps { echo 'Deploy' } }
  }
}
```

Each stage represents one step in the CI/CD process.

- Build → Compiles or builds the application
- Test → Runs automated tests
- Deploy → Deploys the application

6. **Introduce:**

o agent: Defines where the pipeline or a specific stage runs. Use agent any to run on any available agent, agent { label 'linux' } to target a specific agent type, or agent none to let each stage specify its own agent.

o stages: A logical step in the pipeline (e.g., Build, Test, Deploy). Each stage groups related tasks and appears as a visual block in the Jenkins UI, showing progress and status.

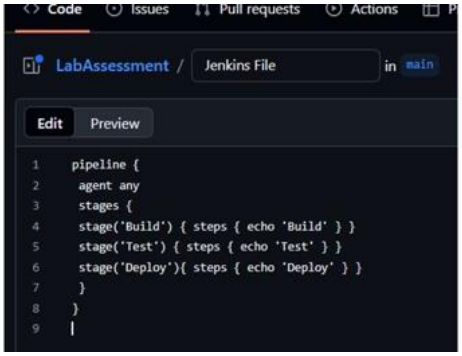
o steps: The actual commands or actions executed within a stage (e.g., sh 'mvn clean install' or bat 'build.bat'). Steps run sequentially and perform tasks like compiling code, running tests, or deploying applications.

Outcome: Learner writes basic pipelines confidently.

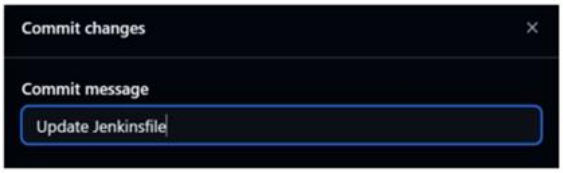
5. Learning Goal: Pipeline as code from repository

Stepwise Tasks

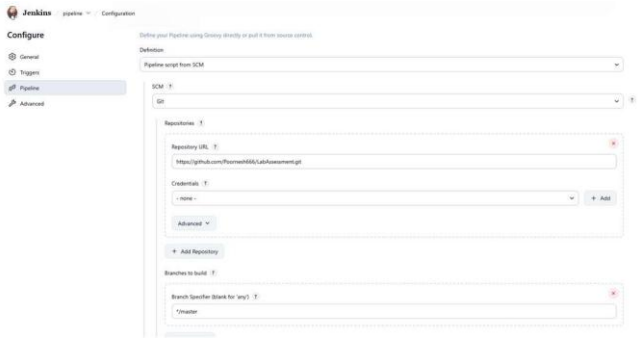
1.**Create Jenkins File**



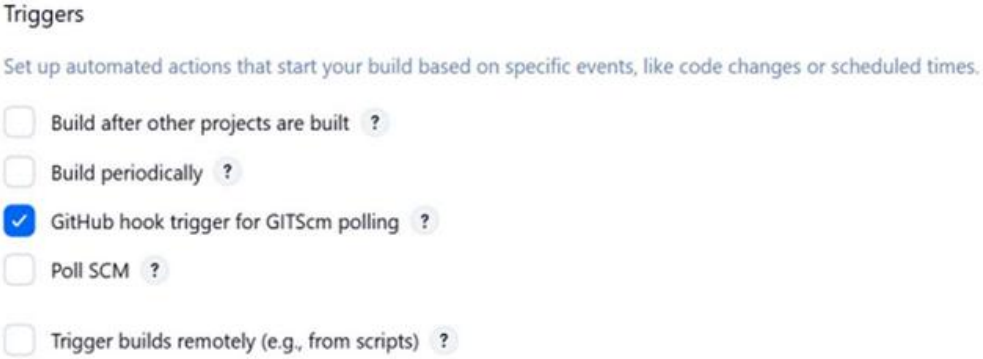
2. Commit Jenkinsfile to GitHub



3. **Configure Pipeline job:** **o SCM → Git**



4.Trigger build automatically



5. **Modify Jenkinsfile and rebuild**

Code Blame 9 lines (9 loc) · 192 Bytes

```
pipeline {
 agent any
 stages {
 stage('Build') { steps { echo 'Build' } }
 stage('Test') { steps { echo 'Test' } }
 stage('Deploy'){ steps { echo 'Deploy' } }
 }
}
```

6. **View Stage View**

Pipeline-From-Git

Stage View

	Declarative: Checkout SCM	Build	Test	Deploy
Average stage times: (full run time: ~7s)	2s	165ms	126ms	131ms
#2 Feb 04 22:35 2 commits	2s	165ms	124ms	124ms
#1 Feb 04 22:30 No Changes	2s	166ms	129ms	138ms

Outcome: Learner practices Infrastructure as Code concept.

****THE END****